Estructuras de Datos Dinámicas
Listas - Pilas

Programador Universitario en Sistemas
Primer Año - 2° cuatrimestre

Estructuras de Datos Dinámicas

En esta clase vamos a trabajar:

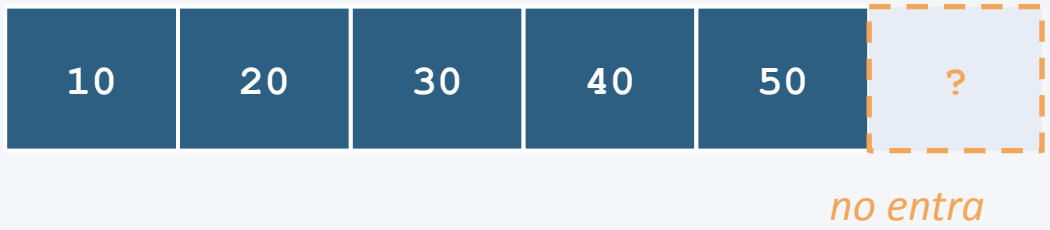
- Limitaciones de los arrays
- Estructuras de datos dinámicas
- Nodos
- Referencias y enlaces entre nodos
- Listas enlazadas
- Operaciones sobre listas
- Complejidad de las operaciones
- Lista vs. Pila

El problema del tamaño

Un array tiene un tamaño determinado en el momento en que se crea, y ese tamaño no cambia mientras el programa se ejecuta. Esto es una decisión que tomamos una sola vez, al principio:

```
int[] numeros = new int[5];
```

Con esta declaración tenemos espacio reservado para exactamente 5 elementos. Si en algún momento del programa necesitamos guardar un sexto valor, no hay dónde ponerlo:

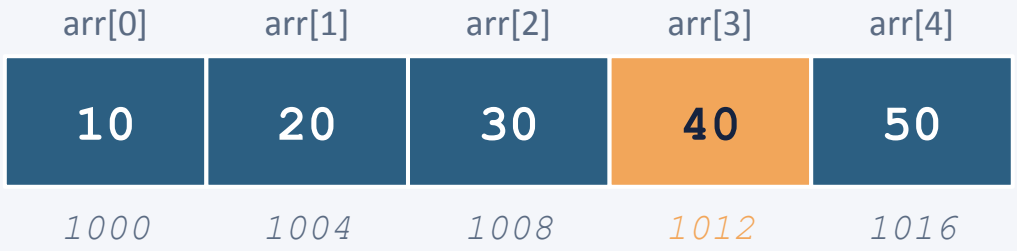


Una salida posible es crear un array más grande, copiar ahí todos los elementos existentes y recién después agregar el nuevo. Funciona, pero tiene un costo: hay que mover TODOS los elementos anteriores, aunque solo queramos agregar uno al final.

¿Por qué mover todos los datos si solamente queremos agregar uno?

¿Por qué acceder a un array es O(1)?

Cuando hacemos arr[3], Java no recorre nada: calcula la dirección de memoria con una sola cuenta.




```
dirección(arr[i])  =  dirección_base  +  i × tamaño_del_tipo
dirección(arr[3])  =  1000  +  3 × 4  =  1012
```

Es una sola cuenta — multiplicación + suma, sin importar si accedemos a arr[3] o a arr[3.000.000]

No hace falta mirar — arr[0], arr[1] ni arr[2] para llegar a arr[3]: por eso el acceso por índice es O(1)

Esto funciona — porque el array reserva memoria contigua: cada elemento está exactamente a "tamaño_del_tipo" bytes del anterior

Pregunta para tirar a la clase: ¿por qué entonces ArrayList.get(i) es rápido, pero LinkedList.get(i) es lento? (La respuesta está en la próxima diapositiva.)

?

Una estructura que pueda crecer

El problema del array no es solo que sea chico: es que su tamaño hay que decidirlo de antemano, antes incluso de saber cuántos datos vamos a manejar realmente. Necesitamos otra forma de pensar la estructura.

Objetivo — una estructura dinámica busca adaptarse a la cantidad de elementos que contiene en cada momento, sin fijar un límite de antemano

No pensamos primero en — "tengo 10 posiciones reservadas, ocupe o no ocupe todas", como pasa con un array declarado de entrada

Pensamos en — "tengo estos elementos ahora, y puedo agregar o quitar elementos a medida que el programa se ejecuta", sin preocuparnos por un límite fijo

La clave — la estructura se adapta durante la ejecución del programa, creciendo y achicándose según haga falta, elemento por elemento

Ejemplo cotidiano — es como una lista de compras en papel: no sabés de antemano cuántos ítems vas a anotar, simplemente agregás una línea más cuando la necesitás

Esta idea de "tamaño variable en tiempo de ejecución" es la que vamos a implementar concretamente en los próximos minutos, con una herramienta muy simple: el nodo.

C

¿Cómo puede crecer?

Si ya no queremos un array de tamaño fijo, tenemos que decidir cómo se va a reservar la memoria para cada elemento nuevo a medida que el programa avanza.

Una posibilidad — reservar de entrada una gran cantidad de memoria "por las dudas", pero eso desperdicia espacio si al final usamos poco, y sigue teniendo un límite máximo

Otra posibilidad, más prolija — crear cada elemento recién cuando lo necesitamos, ni antes ni de más; esto se conoce como reserva dinámica de memoria

Consecuencia — cada elemento puede ocupar un espacio independiente en memoria, no necesariamente uno al lado del otro como en un array

El desafío que aparece — si los elementos ya no están "pegados" en memoria, necesitamos una forma explícita de saber dónde está el siguiente elemento

Esto es distinto a un array — ahí la posición te la da la aritmética de índices; acá hace falta que cada elemento "conozca" al que sigue

Ese "saber dónde está el siguiente" es exactamente el problema que resuelve el nodo, la pieza que vemos en la próxima diapositiva.

N

Aparece el nodo

La pieza que nos permite resolver el problema anterior es una unidad muy simple, que vamos a usar y reutilizar durante el resto de la unidad: el nodo. Todo lo que viene después (listas, pilas, colas, árboles) se construye a partir de esta misma idea.

Un nodo es — una unidad que almacena dos cosas a la vez: un dato y una referencia hacia otro nodo

El dato — representa la información que realmente nos interesa guardar (un número, un nombre, o incluso otro objeto más complejo)

La referencia — no guarda un valor "de contenido", sino la dirección de memoria de otro nodo, y es lo que permite conectarlos entre sí

Por qué importa — cada nodo puede crearse en cualquier lugar de la memoria; la conexión no depende de estar uno al lado del otro

Un nodo sin siguiente — cuando un nodo es el último de la cadena, su referencia "siguiente" simplemente no apunta a nada (null)



Estructura conceptual de un nodo

Analogía: pensalo como un vagón de tren. Cada vagón lleva su propia carga (el dato) y un enganche hacia el vagón siguiente (la referencia). El tren completo se arma encadenando vagones.

Un nodo en Java

Traducimos la idea conceptual del nodo (dato + referencia) a una clase Java, muy simple, que vamos a reutilizar en toda la unidad:

```
class Nodo {
    int dato;
    Nodo siguiente;
}
```

int dato — es el contenido que nos interesa guardar; en un caso real podría ser otro tipo, como String, double, o incluso un objeto de otra clase (Alumno, Producto, etc.)

Nodo siguiente — es la línea clave de toda la clase: un objeto Nodo puede mantener una referencia a otro objeto Nodo, de su mismo tipo

Nada más — la clase no necesita saber nada sobre el resto de la lista; solo conoce al nodo inmediatamente siguiente, y eso alcanza para poder recorrerla completa

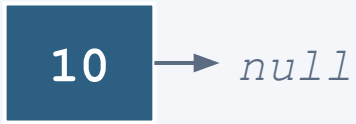
Valor por defecto — si no inicializamos siguiente explícitamente, Java le asigna null automáticamente, que representa "no hay nodo después de este"

Ahí comienza la estructura dinámica: un objeto que se referencia a sí mismo (a otro de su misma clase) es lo que va a permitir encadenar tantos nodos como haga falta.

Conectando nodos

Veamos qué va pasando, paso a paso, a medida que conectamos nodos entre sí usando la referencia "siguiente" que acabamos de definir.

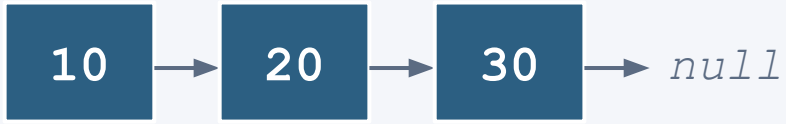
Un nodo aislado



Dos nodos conectados



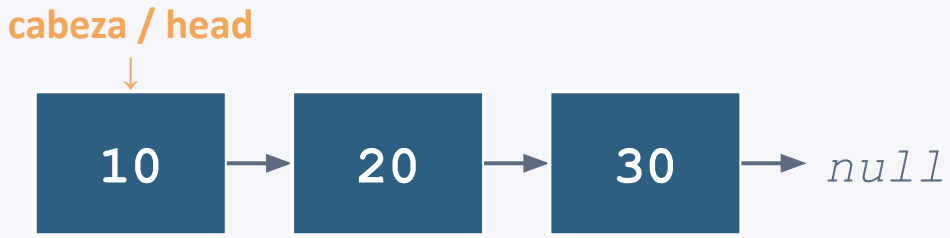
Tres nodos conectados



No necesitamos que los nodos estén uno al lado del otro en memoria: la relación entre ellos está dada por las referencias, no por la cercanía física.

De nodos a lista

Cuando conectamos varios nodos de esta forma, obtenemos una lista enlazada: una secuencia de nodos donde cada uno conoce únicamente al que le sigue.



Lista enlazada — es un tipo de dato que sabe recorrer una secuencia de nodos sin conocerlos a todos de entrada.
cabeza — la lista necesita conocer, como mínimo, dónde comienza. A esa referencia inicial se la suele llamar cabeza (o head).

Desde la cabeza, y siguiendo las referencias de cada nodo, podemos alcanzar cualquier elemento de la lista.

La lista vacía

¿Y si todavía no cargamos ningún elemento? Una lista puede existir perfectamente sin contener ningún nodo.



En Java, esto se representa con una simple asignación:

```
Nodo cabeza = null;
```

Este caso (lista vacía) va a ser importante porque muchas operaciones — insertar, buscar, eliminar — deben saber manejarlo antes de intentar acceder a un nodo que no existe.

¿Qué necesitamos guardar?

Una implementación sencilla de la lista puede tener un único atributo: la referencia al primer nodo.

```
class Lista {
    Nodo cabeza;
}
```

La lista NO necesita guardar todos los nodos — le alcanza con la referencia al primero

Desde cabeza — se puede recorrer el resto de la lista siguiendo, uno a uno, los atributos siguiente de cada nodo

Esto ahorra memoria — no hay una estructura extra por fuera de los propios nodos

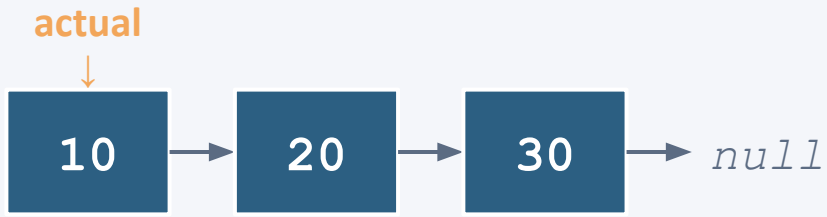
Toda la información de la lista está "desperdigada" entre los nodos; la clase Lista solo guarda el punto de entrada.

Recorrer una lista

Para visitar todos los elementos, necesitamos una variable auxiliar que vaya avanzando de nodo en nodo hasta llegar al final.

```
Nodo actual = cabeza;

while (actual != null) {
    System.out.println(actual.dato);
    actual = actual.siguiente;
}
```

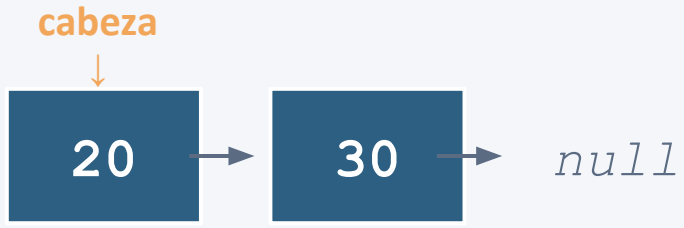


La variable actual va avanzando: 10 → 20 → 30 → null. No necesitamos saber previamente cuántos elementos tiene la lista.

Este patrón — nodo actual + while + reasignar actual — se repite en casi todas las operaciones que vamos a ver hoy.

Primera operación: insertar al inicio

Empezamos con la operación más simple de todas. Partimos de una lista existente y queremos agregar un valor nuevo antes de la cabeza actual.



Queremos insertar 10. Primero creamos el nodo nuevo (todavía desconectado):

```
Nodo nuevo = new Nodo();
nuevo.dato = 10;
```



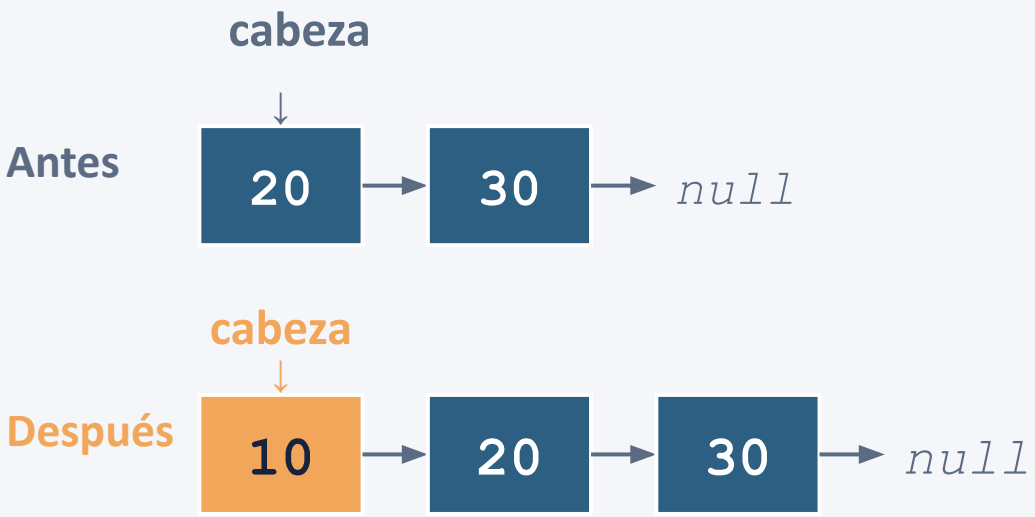
nuevo (aún sin conectar)

Todavía falta un paso: conectar este nodo nuevo con el resto de la lista. Eso lo vemos en la próxima diapositiva.

Insertar al inicio: la conexión

La conexión se hace con solamente dos líneas, que redirigen referencias en lugar de mover datos:

```
nuevo.siguiente = cabeza;
cabeza = nuevo;
```



Solo modificamos referencias: no desplazamos ni copiamos ningún elemento existente de la lista.

1

¿Cuánto cuesta insertar al inicio?

Para responder esto, contamos cuántos pasos hacen falta, sin importar cuántos elementos tenga ya la lista.

Paso 1 — crear el nodo nuevo

Paso 2 — conectar su referencia siguiente hacia la cabeza actual

Paso 3 — cambiar cabeza para que apunte al nodo nuevo

Lo importante — la cantidad de elementos de la lista no cambia la cantidad de pasos: son siempre 3, tenga la lista 2 o 2 millones de nodos

Por eso decimos que la inserción al inicio tiene complejidad O(1): tiempo constante, independiente del tamaño de la lista.

Insertar al final

Ahora la situación es distinta: solo conocemos la cabeza, y queremos agregar un elemento nuevo después del último nodo.




```
Nodo actual = cabeza;

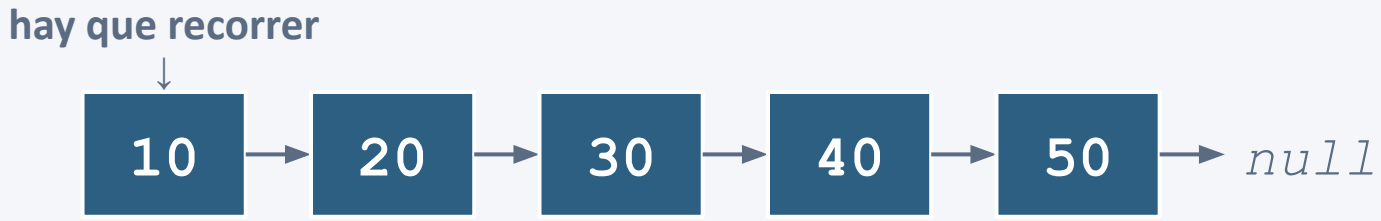
while (actual.siguiente != null) {
    actual = actual.siguiente;
}

actual.siguiente = nuevo;
```

Si solo conocemos cabeza, no hay atajo: hay que recorrer nodo por nodo hasta encontrar el que tiene siguiente == null.

El costo de recorrer

A diferencia de insertar al inicio, para llegar al último nodo hay que visitar todos los anteriores, uno por uno.

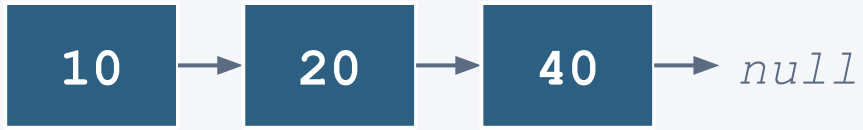


Cuantos más elementos tiene la lista, más nodos hay que visitar. Con cabeza solamente, insertar al final es O(n). Más adelante vemos cómo mejorar esto guardando también una referencia al último nodo.

Insertar en una posición intermedia

El último caso de inserción es agregar un elemento en el medio de la lista, ni al principio ni al final.

Lista actual



Queremos insertar 30 entre 20 y 40. Resultado esperado:

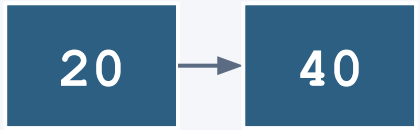


Antes de poder insertar, primero hay que encontrar el nodo anterior al lugar donde queremos insertar (en este caso, el nodo 20).

La idea de la inserción

Una vez que tenemos ubicado el nodo anterior (20), la inserción se resuelve reorganizando únicamente dos referencias.

Antes



Nodo nuevo (creado, sin conectar)



Después



La operación no consiste en mover datos de lugar: consiste en cambiar dos enlaces (referencias siguiente).

Buscar un elemento

Buscar consiste en recorrer la lista desde el principio, comparando el dato de cada nodo con el valor buscado.

```
Nodo actual = cabeza;

while (actual != null) {
    if (actual.dato == 40) {
        return actual;
    }
    actual = actual.siguiente;
}
```

Recorremos — desde cabeza, nodo por nodo, comparando en cada paso

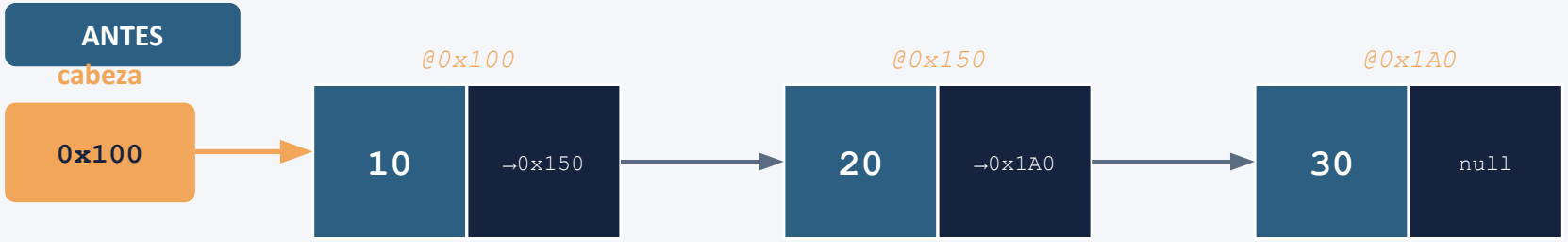
Si encontramos el valor — devolvemos ese nodo inmediatamente, sin seguir recorriendo

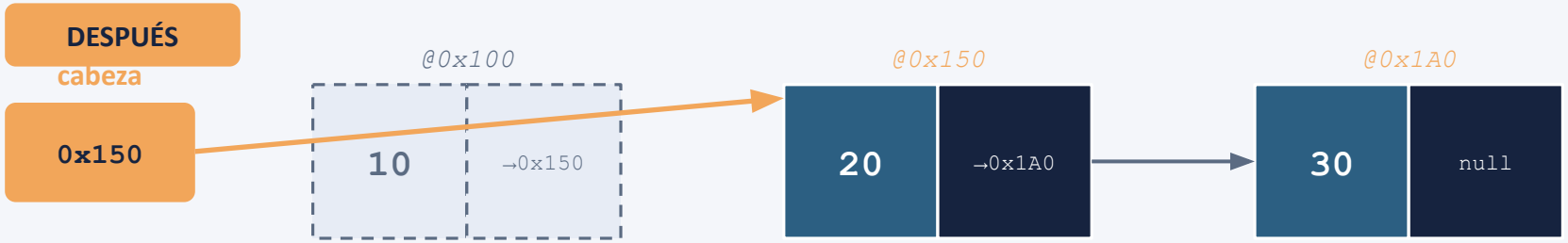
Si llegamos a null — sin haberlo encontrado, el elemento no está en la lista

En el peor caso (el valor no está, o está al final) hay que recorrer toda la lista: búsqueda → O(n).

cabeza = cabeza.siguiente — ¿qué pasa en memoria?

Una referencia no es más que una dirección guardada en una variable. Veamos qué cambia, dirección por dirección.





nadie lo referencia → el GC lo va a liberar

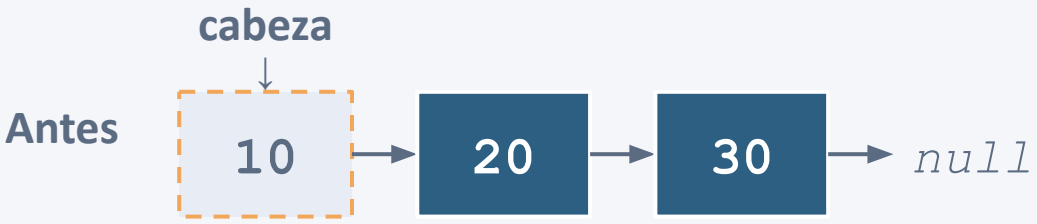
cabeza guarda una dirección, no un nodo.

Antes: cabeza vale 0x100 (apunta al nodo con 10). Al ejecutar cabeza = cabeza.siguiente, leemos el campo siguiente del nodo actual (0x150) y ESE valor pasa a ser el nuevo contenido de cabeza.

El nodo de 0x100 no se borra ni se mueve: simplemente deja de tener alguna variable que lo señale.

Eliminar el primer elemento

Para eliminar el primer nodo de la lista, alcanza con redirigir la cabeza al segundo nodo.



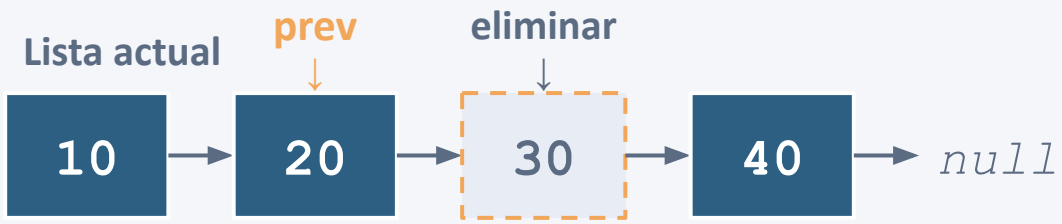

```
cabeza = cabeza.siguiente;
```



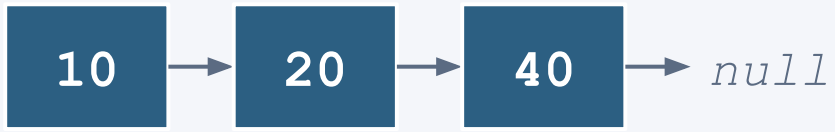
El nodo 10 deja de formar parte de la lista (nadie lo referencia). Eliminar al inicio → O(1), igual que insertar al inicio.

Eliminar un elemento intermedio

Para eliminar un nodo del medio, primero hay que llegar al nodo anterior — no al nodo que queremos borrar.



Después de reconectar:



Igual que al insertar: la eliminación consiste en modificar una referencia (prev.siguiente), no en mover elementos.



El recolector de basura (GC) y las referencias

En Java no liberamos memoria a mano (como el free() de C): el recolector de basura libera automáticamente todo objeto que ya nadie referencia.

Regla del GC — un objeto sigue "vivo" mientras exista al menos una referencia alcanzable que apunte a él (una variable, un atributo de otro objeto vivo, etc.)

Lo que vimos en la diapositiva anterior — al desenganchar el primer nodo, ese nodo queda sin ninguna referencia y pasa a ser basura para el GC

El caso extremo — cabeza = null de golpe deja inalcanzables TODOS los nodos de la lista, no solo el primero: es la única puerta de entrada a toda la cadena

Por qué importa — en una lista simplemente enlazada, cada nodo es referenciado solo por el anterior; perder un eslabón temprano puede desconectar todo lo que sigue

Pregunta!! : en una lista DOBLEMENTE enlazada, ¿alcanza con que una sola variable externa referencie a un nodo del medio para mantener viva a toda la lista?

Los casos que siempre debemos pensar

Cada operación que implementemos sobre una lista tiene que contemplar algunos escenarios límite, además del caso "normal".

Lista vacía — cabeza es null; muchas operaciones deben verificar esto antes de acceder a un atributo

Un solo elemento — insertar o eliminar puede afectar simultáneamente a cabeza y al único nodo existente

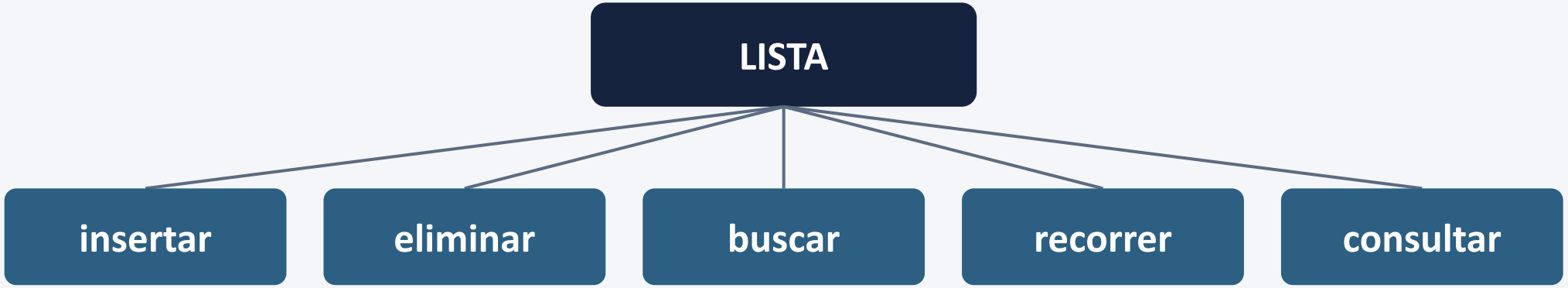
Insertar o eliminar al inicio — siempre implica tocar la variable cabeza

Insertar o eliminar al final — siempre implica llegar hasta el nodo cuyo siguiente es null

La estructura de datos no termina en el caso "normal": los casos límite son parte central de una implementación correcta, no un detalle opcional.

Operaciones fundamentales de una lista

Podemos pensar el TAD (Tipo Abstracto de Dato) Lista a través del conjunto de operaciones que ofrece, sin importar todavía cómo está implementada por dentro.



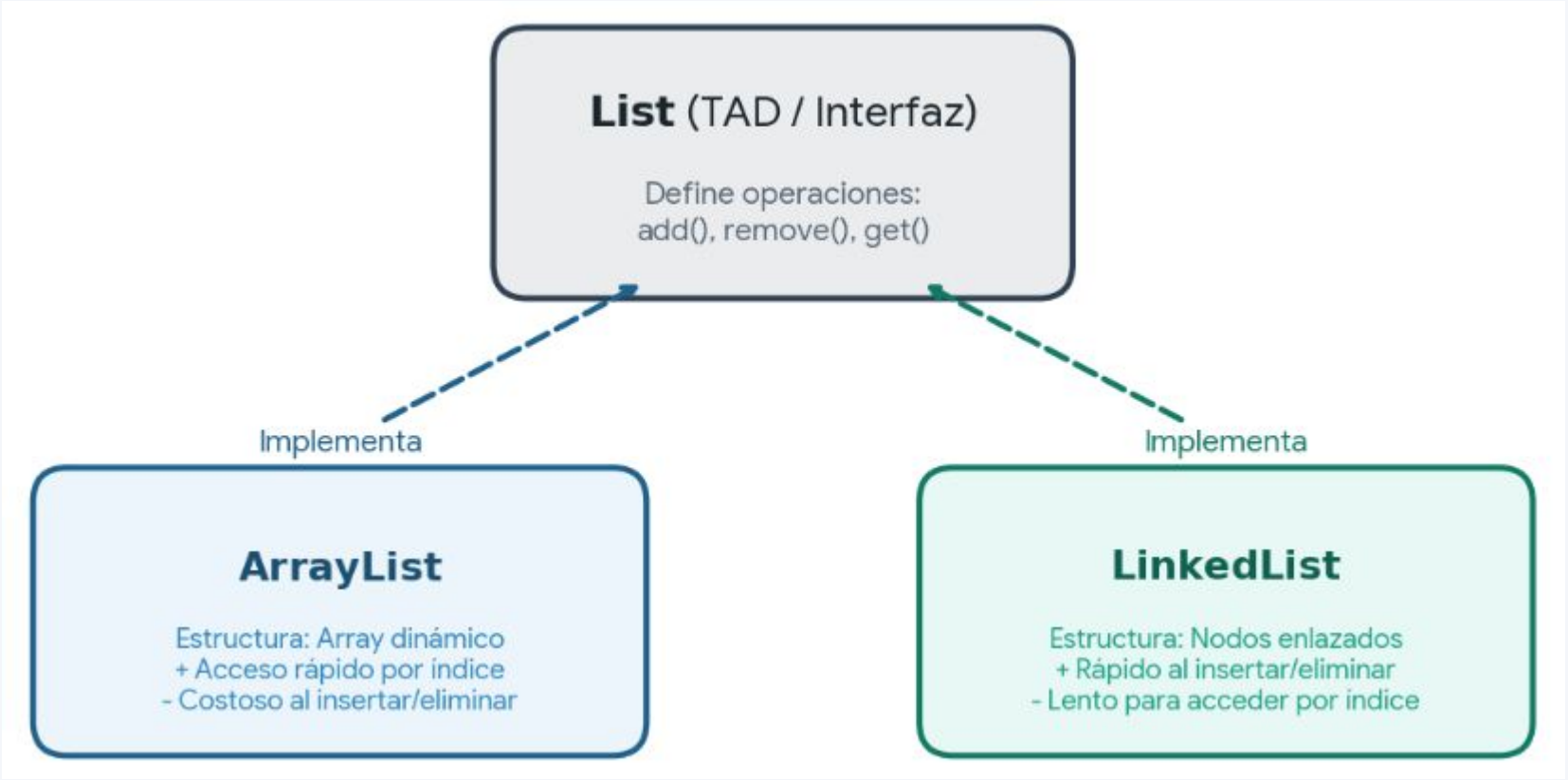
Lo que interesa desde el punto de vista del TAD es qué operaciones ofrece y qué comportamiento tienen — la estructura interna (nodos, referencias) puede cambiar sin afectar esta interfaz.

TAD (Tipo Abstracto de Dato)

List — interfaz que representa una colección de elementos ordenados y permite acceder a ellos por posición.

ArrayList — implementación basada en un array dinámico. Es una opción habitual cuando necesitamos acceso rápido por índice.

LinkedList — implementación basada en nodos enlazados. También implementa List y permite operaciones propias de una lista enlazada.



Java ya tiene Listas

No siempre hace falta implementar una lista desde cero: Java moderno ofrece una interfaz lista para usar.

```
List<Integer> numeros = new ArrayList<>();
o
List<Integer> numeros = new LinkedList<>();

numeros.add(10);
numeros.add(20);
numeros.add(30);

lista.add(10); // agrega un elemento
lista.get(0); // consulta un elemento
lista.remove(0); // elimina un elemento
lista.isEmpty(); // ¿está vacía?
lista.size(); // cantidad de elementos
```

NullPointerException: el error más común al empezar

Este código compila perfecto. El problema aparece recién en tiempo de ejecución, con la lista vacía.

```
public int primerElemento() {
    return cabeza.dato;
    // ¿y si cabeza es null?
}
```

↑ *código que compila sin errores*

```
Exception in thread "main"
java.lang.NullPointerException:
  Cannot read field "dato" because
  "cabeza" is null
    at Lista.primerElemento(Lista.java:15)
    at Main.main(Main.java:8)
```

↑ *lo que se ve recién al ejecutar, con lista vacía*

La corrección: preguntar primero, devolver algo (o avisar) si está vacía.

```
public int primerElemento() {
    if (cabeza == null) {
        throw new RuntimeException("La lista está vacía");
    }
    return cabeza.dato;
}
```

El hábito a instalar: antes de escribir cualquier método nuevo sobre la lista, preguntarse "¿qué pasa si cabeza es null cuando entro acá?" — y recién después programar el resto.

Esto conecta con los casos límite que vimos antes: el NullPointerException es lo que pasa, en la práctica, cuando no se contempló la lista vacía.

Complejidad de una lista enlazada

Operación	Lista con cabeza
Insertar al inicio	O(1)
Eliminar al inicio	O(1)
Buscar	O(n)
Recorrer	O(n)
Insertar al final	O(n)
Eliminar un elemento	O(n)

¿Y si guardamos también el último nodo?

Vimos que insertar al final cuesta O(n) porque hay que recorrer toda la lista. Podemos evitarlo agregando una segunda referencia.

```
class Lista {
    Nodo cabeza;
    Nodo cola;
}

// Insertar al final ahora es directo:
cola.siguiente = nuevo;
cola = nuevo;
```

cola (o tail) — guarda una referencia directa al último nodo de la lista, sin necesidad de recorrerla

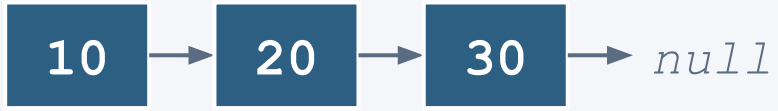
Costo — actualizar cola en cada inserción al final es un paso extra, pero constante

Con esta referencia adicional, insertar al final pasa de O(n) a O(1). Es un ejemplo clásico de "guardar un poco más de información para ganar velocidad".

La lista no siempre tiene una sola forma

Todo lo que vimos hasta ahora corresponde a la lista simplemente enlazada, pero existen variantes que resuelven necesidades distintas.

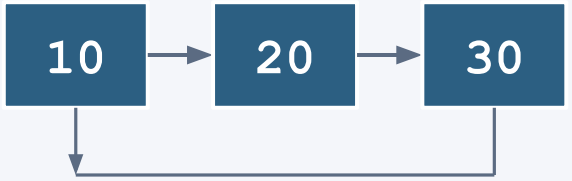
Simplemente enlazada



Doblemente enlazada



Circular



La idea fundamental se mantiene igual en las tres variantes: los nodos se relacionan mediante referencias, no por posición física en memoria.

De la lista a la pila

Ahora cambiamos la pregunta. Ya sabemos armar y recorrer una lista libremente. ¿Qué pasa si le imponemos una regla estricta?



Regla nueva: solo podemos agregar y quitar elementos por un extremo.

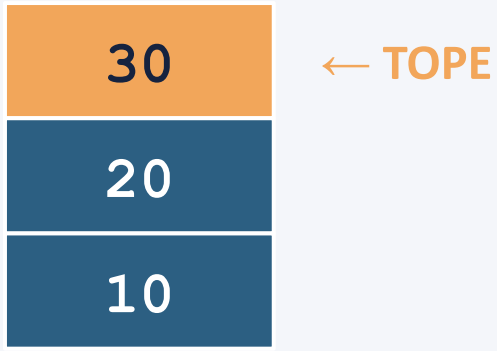
Acabamos de restringir las operaciones de la lista. Esa estructura, con ese comportamiento específico, tiene nombre propio:

Pila

Una pila: LIFO

Una pila sigue una única regla de acceso: LIFO — Last In, First Out. El último elemento que entra es el primero que sale.

```
push(10);
push(20);
push(30);
```



Si ahora ejecutamos pop(), sale 30 (el último que entró).

LIFO es la regla que define a la pila. Todo lo que sigue (push, pop, peek) existe para respetarla.

Pensarlo con algo cotidiano

Antes de ver código, conviene tener una imagen mental simple y muy conocida: una pila de platos.



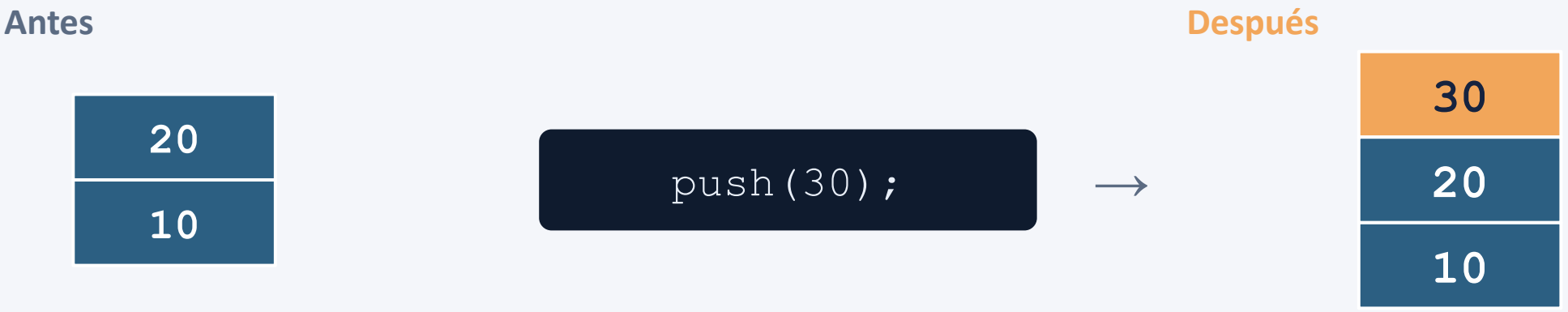
No sacamos el plato del medio: agregamos y quitamos siempre por el mismo extremo. Ese extremo es lo que llamamos el tope de la pila.

Las tres operaciones clave de una pila

1	**push**	Agregar un elemento en el tope
2	**pop**	Quitar y devolver el elemento del tope
3	**peek**	Consultar el elemento del tope, sin quitarlo

Push

push agrega un elemento nuevo, y ese elemento pasa a ser el nuevo tope de la pila.

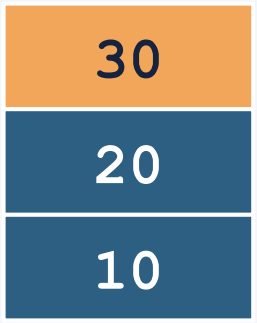


En una implementación con nodos, agregar el nuevo nodo al inicio de la cadena es exactamente lo mismo que insertar al inicio de una lista: push es O(1).

Pop

pop quita el elemento que está en el tope y lo devuelve como resultado.

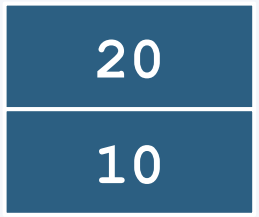
Antes




```
pop();
```

→ *sale 30*

Después

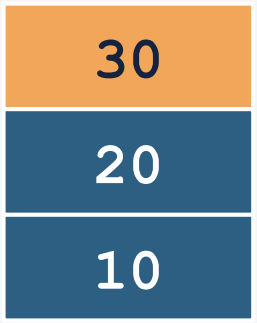


Si el tope está en el inicio de la lista de nodos, quitar el primer nodo es la misma operación que eliminar al inicio: pop es O(1).

Peek

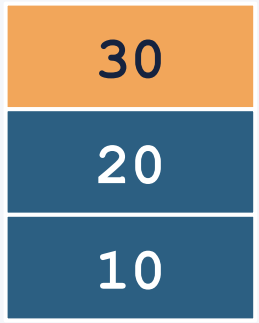
peek consulta cuál es el valor del tope, pero a diferencia de pop, no modifica la pila.

Antes



peek() → devuelve 30, y la pila queda exactamente igual

Después



peek consulta, no elimina. Es útil, por ejemplo, para "mirar" el próximo paso a deshacer sin deshacerlo todavía.

¿Qué pasa si la pila está vacía?

Antes de hacer pop() o peek(), una buena implementación siempre debería preguntarse si hay algo para sacar.

Pila vacía — el tope no apunta a ningún nodo (tope == null)

El problema — no podemos hacer pop() sobre una pila vacía; no hay ningún elemento que devolver

La solución — toda implementación de pila debería ofrecer un método isEmpty(), que se consulta antes de operar

En código — la verificación es tan simple como if (tope == null) { /* pila vacía */ }

Ignorar este caso es una de las causas más comunes de errores en tiempo de ejecución al trabajar con pilas.

Pila implementada con nodos

Podemos reutilizar exactamente la misma idea del nodo que usamos para la lista, cambiando solo el nombre del atributo.

```
class Nodo {
    int dato;
    Nodo siguiente;
}

class Pila {
    Nodo tope;
}
```

La estructura interna — es idéntica a la de una lista: nodos conectados por referencias

Lo que cambia — no es el nodo, sino la regla de acceso: solo se opera sobre tope

La lista y la pila comparten la misma idea de implementación por dentro; lo que las distingue es qué operaciones permitimos hacer desde afuera.

Pila ≠ lista: mismo material, distinta regla

	Lista	Pila
Operaciones permitidas	insertar, eliminar, buscar, recorrer (en cualquier posición)	push, pop, peek (solo sobre el tope)
Acceso a elementos	Cualquier posición de la secuencia	Únicamente el elemento del tope
Regla de orden	No impone ningún orden particular	LIFO obligatorio
Eliminar del medio	Es una operación soportada	No tiene sentido: rompería la regla LIFO

La pila también puede implementarse con un array

No es obligatorio usar nodos: la misma idea de pila puede construirse sobre un array con un índice que marque el tope.

```
int[] elementos;
int tope;
```



Tenemos entonces dos enfoques válidos para implementar una pila: con array o con nodos. La estructura lógica (LIFO) es la misma; la implementación por dentro es distinta.

¿Qué cambia entre las implementaciones?

	Con array	Con nodos
Ubicación en memoria	Posiciones contiguas	Nodos dispersos, unidos por referencias
Tamaño	Fijo (o hay que redimensionar)	Crece y decrece dinámicamente
push / pop / peek	Operan sobre el índice tope	Operan sobre el nodo tope
Complejidad de push/pop	O(1) amortizado	O(1)

Java ya tiene pilas

No siempre hace falta implementar una pila desde cero: Java moderno ofrece una interfaz lista para usar.

```
Deque<Integer> pila = new ArrayDeque<>();

pila.push(10);
pila.push(20);
pila.push(30);

pila.peek();      // consulta el tope
pila.pop();       // quita y devuelve el tope
pila.isEmpty();   // ¿está vacía?
```

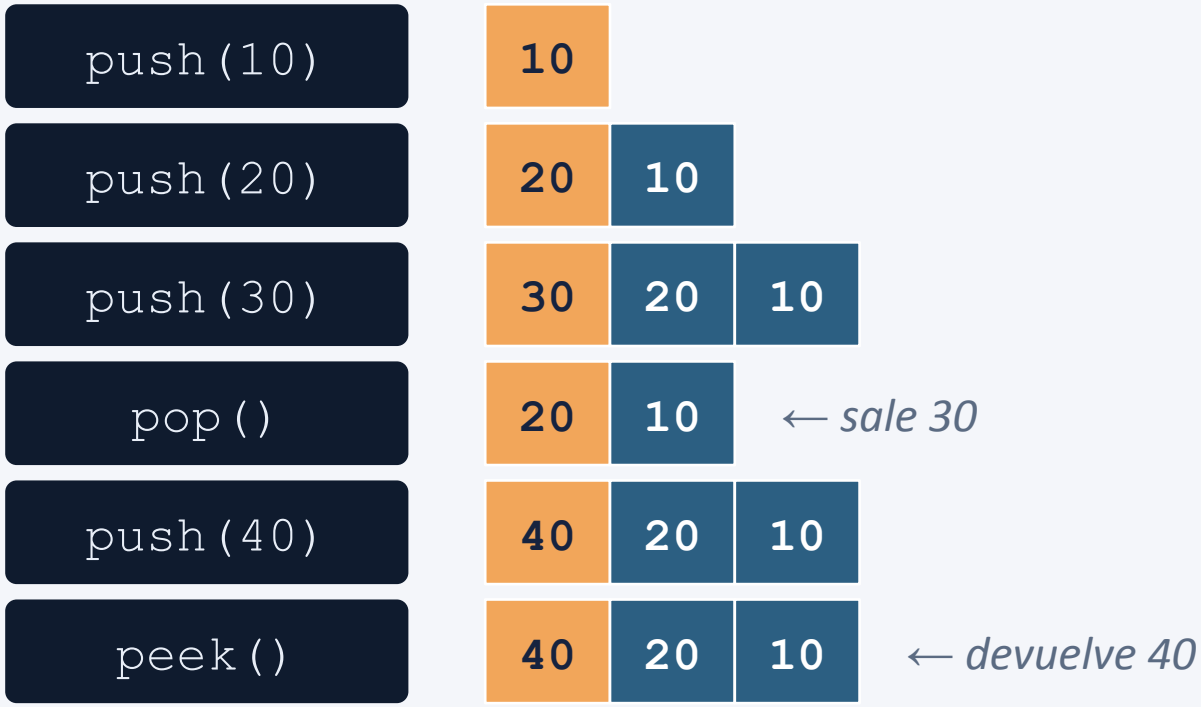
Deque — significa "double-ended queue"; se puede usar como pila (push/pop) o como cola

ArrayDeque — es la implementación recomendada en Java moderno para pilas, por encima de la vieja clase

La biblioteca estándar ya resuelve todo lo que armamos hoy "a mano": nodos, referencias, casos límite.

Una secuencia completa

Para cerrar la parte de pilas, sigamos paso a paso una secuencia de operaciones sobre una pila que arranca vacía.



En todo momento, el último elemento agregado es el que está arriba, listo para ser el próximo en salir.

¿Dónde aparecen las pilas?

La regla LIFO no es solo un ejercicio de programación: aparece constantemente en problemas informáticos reales.

Deshacer acciones (Ctrl+Z)

Cada acción se apila; deshacer siempre saca primero la última acción realizada.

Navegación web (Atrás)

Las páginas visitadas se apilan; "volver" te lleva a la última que visitaste.

Evaluación de expresiones

Los paréntesis y operadores de expresiones como (A + B) * C se resuelven con una pila.

Llamadas de métodos

Cuando un método llama a otro, la ejecución también utiliza internamente una estructura de pila.

Lista y pila: misma idea, distinta regla

Para cerrar la comparación, veamos el árbol de decisiones completo: de una estructura general a dos comportamientos distintos.



La diferencia fundamental no está en cómo guardamos los datos (en ambas usamos nodos), sino en qué operaciones permitimos realizar y en qué orden.

El recorrido de hoy

Repasemos el camino completo que hicimos, desde una limitación muy concreta hasta dos estructuras de datos nuevas.

ARRAY	→	**tamaño limitado**	*Empezamos con una necesidad: ¿qué hacemos cuando no sabemos cuántos elementos vamos a necesitar?*
NODO → REFERENCIAS	→	**LISTA ENLAZADA**	*Buscamos una estructura que pudiera crecer, y llegamos al nodo: dato + referencia al siguiente.*
LISTA + acceso restringido	→	**PILA (LIFO)**	*Restringimos el comportamiento de la lista a un solo extremo, y obtuvimos la pila.*

Cada estructura nueva no reemplazó a la anterior: se construyó reutilizando la misma pieza básica, el nodo.

Para la práctica

Antes de la próxima clase, dos desafíos para implementar y comparar con las herramientas nativas de Java.

Desafío 1 — Lista

- Implementar class Lista con nodos propios
- insertarInicio() e insertarFinal()
- buscar() y eliminar()
- mostrar() — recorrer e imprimir todos los elementos

Desafío 2 — Pila

- Implementar class Pila con nodos propios
- push(), pop() y peek()
- isEmpty() — contemplar el caso de pila vacía
- Comparar después con ArrayDeque de Java